

Analisis Arsitektur dan Kinerja Modifikasi IP Core MC8051 pada Eksekusi Program Array Summation

Hizkia Imanuel Prasetya Fikri Abqari Fawwaz M. Daffa Raditya Budiman Samuel Marcel Jonathan Panjaitan
NIM: 24/533897/PA/22627 NIM: 24/537606/PA/22785 NIM: 24/537800/PA/22797 NIM: 24/541509/PA/22977

Muhammad Alfarizq
NIM: 24/542274/PA/23023

Abstract—Proyek ini mendemonstrasikan eksekusi program manipulasi memori (Array Summation) menggunakan platform simulator IP Core MC8051 dari Oregano Systems versi 1.6. Analisis dilakukan secara mendalam dari sudut pandang organisasi dan arsitektur komputer dengan mengamati transisi sinyal digital pada simulator ModelSim. Selain menguji fungsionalitas program dasar, proyek ini juga melakukan eksperimen rekonfigurasi pada file VHDL untuk memodifikasi struktur internal mikrokontroler, yaitu dengan mematikan Serial Interface Unit (SIU) dan mengurangi jumlah Timer/Counter. Hasil modifikasi menunjukkan perubahan arsitektur internal yang signifikan tanpa mengganggu fungsionalitas eksekusi unit logika aritmatika utama. Perbandingan performa antara konfigurasi standar dan modifikasi disajikan secara kuantitatif, mencakup jumlah siklus mesin, waktu eksekusi, dan jejak sinyal kendali. Seluruh kode, konfigurasi, dan file pendukung telah didokumentasikan melalui repositori GitHub.

Index Terms—MC8051, Oregano Systems, Arsitektur Komputer, VHDL, ModelSim, Loop Unrolling, Rekonfigurasi Periferal

I. PENDAHULUAN

Mikrokontroler memegang peranan krusial dalam sistem tertanam modern, mulai dari perangkat Internet of Things (IoT) hingga sistem kendali industri. Arsitektur Intel 8051, meskipun diperkenalkan sejak tahun 1980, tetap menjadi fondasi penting dalam pembelajaran organisasi dan arsitektur komputer. Hal ini disebabkan oleh set instruksinya yang bersifat CISC (Complex Instruction Set Computer) dengan lebih dari 255 opcode, serta struktur memorinya yang memisahkan ruang instruksi dan data secara fisik—sebuah implementasi murni dari arsitektur Harvard. Pemisahan bus instruksi dan bus data ini memungkinkan akses simultan terhadap kedua jenis memori, yang merupakan konsep fundamental dalam desain prosesor modern.

Dalam makalah ini, kami memilih menggunakan platform Oregano MC8051 versi 1.6, sebuah IP (Intellectual Property) Core open-source berbasis VHDL yang dapat disimulasikan secara fungsional pada tingkat Register-Transfer Level (RTL). Berbeda dengan emulator perangkat lunak yang hanya

mensimulasikan perilaku instruksi, Oregano MC8051 memungkinkan pengamatan langsung terhadap aliran sinyal digital di setiap siklus detak jam (clock cycle), sehingga sangat ideal untuk memahami bagaimana komponen-komponen perangkat keras prosesor bekerja secara bersinergi.

Proyek ini memiliki tiga tujuan utama. Pertama, menganalisis bagaimana sebuah prosesor mengeksekusi instruksi per instruksi di tingkat perangkat keras, mulai dari pengambilan (fetch), dekode, hingga eksekusi operasi aritmatika. Kedua, membandingkan performa eksekusi antara program dengan struktur loop konvensional dan program yang dioptimasi menggunakan teknik loop unrolling. Ketiga, melakukan eksperimen rekonfigurasi arsitektur dengan mematikan periferal yang tidak digunakan untuk mengevaluasi dampaknya terhadap jejak sinyal kendali dan efisiensi desain logika.

Repositori kode sumber, program assembly, file VHDL modifikasi, dan dokumentasi pendukung dari proyek ini dapat diakses secara publik melalui tautan GitHub berikut: <https://github.com/Alfarizq2024/Analisis-Arsitektur-dan-Kinerja-Modifikasi-IP-Core-MC8051-pada-Eksekusi-Program-Array-Summation>

II. TINJAUAN PLATFORM: OREGANO MC8051

A. Deskripsi Platform

MC8051 IP Core dari Oregano Systems adalah deskripsi perangkat keras tingkat RTL (Register-Transfer Level) yang mereproduksi komponen fisik dari mikrokontroler Intel 8051 dalam bahasa deskripsi perangkat keras VHDL. Implementasi ini bersifat siklus-akurat (cycle-accurate), artinya perilaku sinyal keluaran pada setiap clock cycle identik dengan perilaku perangkat keras asli. Platform ini dikembangkan oleh Oregano Systems dan dirilis dalam versi 1.6 dengan lisensi open-source, sehingga memungkinkan modifikasi dan eksperimen mendalam terhadap arsitektur internal prosesor.

Berbeda dengan simulator perangkat lunak tingkat tinggi (seperti EdSim51 atau Proteus), Oregano MC8051 mensimulasikan setiap gate logika dan flip-flop yang membentuk prosesor. Ini berarti pengguna dapat mengamati aliran sinyal

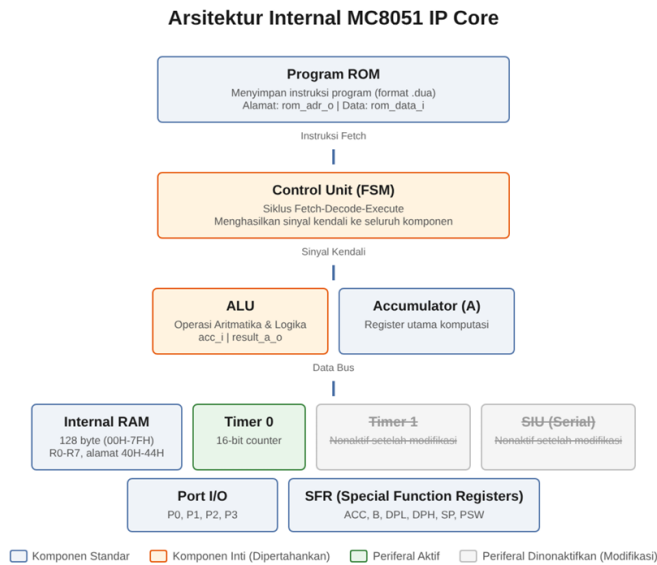


Fig. 1: Arsitektur internal MC8051 IP Core. Komponen berwarna oranye merupakan unit inti yang tetap aktif, sedangkan komponen berwarna abu-abu dengan coret merupakan periferal yang dinonaktifkan pada konfigurasi modifikasi.

digital (0 dan 1) di setiap siklus detak jam, termasuk sinyal-sinyal internal yang tidak terlihat pada simulator perangkat lunak, seperti transisi state pada Finite State Machine (FSM) Control Unit, perubahan nilai pada bus internal ALU, serta aktivitas baca/tulis pada memori RAM internal.

Platform ini terdiri dari beberapa modul VHDL utama yang merepresentasikan komponen fungsional prosesor. Modul-modul tersebut diorganisasi secara hierarkis, dimulai dari komponen-komponen dasar seperti ALU core dan Control FSM, hingga entitas top-level yang mengintegrasikan seluruh komponen menjadi sistem yang utuh. Struktur hierarkis ini memungkinkan modifikasi pada tingkat komponen tanpa harus mengubah keseluruhan desain.

B. Arsitektur Internal

Arsitektur internal MC8051 IP Core terdiri dari beberapa komponen utama yang saling terhubung melalui bus data dan bus alamat. Gambar 1 menampilkan diagram arsitektur internal MC8051 yang menunjukkan interkoneksi antar komponen, termasuk komponen periferal yang akan dimodifikasi dalam eksperimen ini.

Komponen inti yang bertanggung jawab atas eksekusi instruksi meliputi:

Control Unit (FSM): Berfungsi sebagai pengoordinasi utama siklus instruksi. Control Unit mengimplementasikan Finite State Machine yang menghasilkan sinyal-sinyal kendali ke seluruh komponen prosesor berdasarkan instruksi yang sedang dieksekusi. Setiap instruksi 8051 memerlukan 1 hingga 4 siklus mesin (machine cycle), di mana setiap siklus mesin terdiri dari 12 clock cycle pada implementasi standar, atau 2 clock cycle per siklus mesin pada implementasi MC8051 yang dioptimasi.

ALU (Arithmetic Logic Unit): Menangani operasi aritmatika murni (penjumlahan, pengurangan, perkalian, pembagian) dan operasi logika (AND, OR, XOR, NOT, rotasi). ALU menerima input dari Accumulator dan register B, kemudian menghasilkan output beserta flag-flag status (Carry, Auxiliary Carry, Overflow).

Memory Interface: Terdiri dari ROM interface yang menangani pengambilan instruksi dari memori program, dan RAM interface yang mengelola akses ke memori data internal sebesar 128 byte. Pemisahan interface ini merepresentasikan implementasi arsitektur Harvard, di mana bus instruksi dan bus data bersifat independen.

Periferal: Meliputi Serial Interface Unit (SIU) untuk komunikasi serial UART, Timer/Counter untuk pewaktuan dan pencacahan, serta empat port I/O 8-bit (P0-P3). Periferal-periferal ini bersifat opsional dan dapat dikonfigurasi melalui parameter generic dalam file `mc8051_p.vhd`.

C. Relevansi dengan Arsitektur Komputer

Penggunaan Oregano MC8051 sangat relevan dalam konteks pembelajaran organisasi dan arsitektur komputer karena memperlihatkan wujud fisik (dalam bentuk deskripsi VHDL) dari komponen-komponen penyusun prosesor. Melalui simulasi ModelSim, mahasiswa dapat mengamati secara langsung bagaimana konsep-konsep teoritis diimplementasikan pada tingkat perangkat keras, antara lain: (1) siklus fetch-decode-execute yang merupakan dasar dari arsitektur Von Neumann dan Harvard, (2) peran Program Counter dalam menentukan alamat instruksi berikutnya, termasuk perilakunya saat terjadi percabangan (branch), (3) interaksi antara Accumulator dan ALU dalam menyelesaikan operasi komputasi, (4) manajemen memori yang memisahkan ruang instruksi dan data, serta (5) pengaruh konfigurasi periferal terhadap jejak logika dan efisiensi desain.

III. PROGRAM YANG DIJALANKAN

Kami merancang dua varian program Array Summation berbasis Assembly 8051. Alih-alih operasi linear sederhana, program ini menggunakan metode perulangan (looping) untuk menjumlahkan isi dari 5 blok memori yang berurutan. Varian kedua menggunakan teknik loop unrolling untuk mengurangi overhead instruksi percabangan.

A. Program Utama: Array Summation dengan Loop

Program utama mengisi RAM internal mulai dari alamat 40H hingga 44H dengan data arbitrer (5, 10, 3, 7, 1), kemudian menggunakan pointer R0 untuk menunjuk memori dan instruksi perulangan DJNZ (Decrement and Jump if Not Zero) untuk mengontrol akumulasi data ke dalam Accumulator. Potongan kode program utama ditunjukkan pada Listing 1.

Listing 1: Program Array Summation dengan loop DJNZ.

```

1 ; Inisialisasi data di RAM
2 MOV 40H, #05H ; Memori[40H] = 5
3 MOV 41H, #0AH ; Memori[41H] = 10
4 MOV 42H, #03H ; Memori[42H] = 3
5 MOV 43H, #07H ; Memori[43H] = 7
6 MOV 44H, #01H ; Memori[44H] = 1

```

```

8
9 ; Setup pointer dan counter
10 MOV R0, #40H ; R0 = pointer ke alamat awal
11 MOV R7, #05H ; R7 = counter perulangan
12 MOV A, #00H ; A = Accumulator = 0
13
14 ; Loop penjumlahan
15 LOOP:
16 ADD A, @R0 ; A = A + Memori[R0]
17 INC R0 ; Increment pointer
18 DJNZ R7, LOOP ; R7--, lompat jika R7 != 0
19
20 ; Simpan hasil
21 MOV 50H, A ; Hasil = 1AH (26 desimal)
22 SJMP $ ; Halt

```

Alur eksekusi program adalah sebagai berikut: setelah inisialisasi data, register R0 diatur untuk menunjuk ke alamat awal array (40H), register R7 berfungsi sebagai pencacah perulangan dengan nilai awal 5, dan Accumulator di-reset ke 0. Pada setiap iterasi, instruksi `ADD A, @R0` menambahkan nilai pada alamat yang ditunjuk R0 ke Accumulator, kemudian `INC R0` memajukan pointer ke alamat berikutnya, dan `DJNZ R7, LOOP` mengurangi nilai R7 dan melakukan lompatan balik jika R7 belum bernilai nol. Setelah lima iterasi, Accumulator berisi hasil penjumlahan $5+10+3+7+1 = 26$ (1AH dalam heksadesimal), yang kemudian disimpan ke alamat memori 50H.

B. Program Optimasi: Loop Unrolling

Untuk eksperimen perbandingan performa, kami juga merancang versi teroptimasi menggunakan teknik loop unrolling. Teknik ini menghilangkan instruksi `DJNZ` dan `INC R0` yang menambah overhead pada setiap iterasi loop, menggantinya dengan lima instruksi `ADD` berurutan yang mengakses alamat memori secara langsung. Potongan kode program teroptimasi ditunjukkan pada Listing 2.

Listing 2: Program Array Summation dengan loop unrolling.

```

1 ; Inisialisasi data (sama dengan versi loop)
2 MOV 40H, #05H
3 MOV 41H, #0AH
4 MOV 42H, #03H
5 MOV 43H, #07H
6 MOV 44H, #01H
7
8 ; Penjumlahan langsung (tanpa loop)
9 MOV A, 40H ; A = 5
10 ADD A, 41H ; A = 5 + 10 = 15
11 ADD A, 42H ; A = 15 + 3 = 18
12 ADD A, 43H ; A = 18 + 7 = 25
13 ADD A, 44H ; A = 25 + 1 = 26
14
15 ; Simpan hasil
16 MOV 50H, A ; Hasil = 1AH (26 desimal)
17 SJMP $ ; Halt

```

Dengan teknik loop unrolling, overhead instruksi percabangan (`DJNZ`) dan increment pointer (`INC R0`) yang semula dieksekusi lima kali kini dihilangkan sepenuhnya. Hal ini mengurangi jumlah total instruksi dari 18 instruksi menjadi 13 instruksi (tidak termasuk inisialisasi data yang identik), sehingga berpotensi mengurangi jumlah clock cycle yang diperlukan untuk menyelesaikan komputasi.

C. Tabel Perbandingan Instruksi

Tabel I menyajikan perbandingan kuantitatif antara kedua varian program dari segi jumlah instruksi dan estimasi clock

TABLE I: Perbandingan instruksi antara versi loop dan loop unrolling.

Parameter	Loop (DJNZ)	Loop Unrolling
Jumlah instruksi (total)	18	13
Instruksi inisialisasi	8	8
Instruksi komputasi	10	5
Overhead loop (DJNZ+INC)	10	0
Estimasi siklus mesin	23	13
Estimasi clock cycle	46	26

cycle. Estimasi clock cycle dihitung berdasarkan jumlah siklus mesin per instruksi sesuai datasheet Intel 8051, di mana instruksi `MOV direct, #data` memerlukan 1 siklus mesin, `ADD A, @R0` memerlukan 1 siklus mesin, `INC R0` memerlukan 1 siklus mesin, `DJNZ Rn, rel` memerlukan 2 siklus mesin, dan `ADD A, direct` memerlukan 1 siklus mesin.

Dari tabel tersebut terlihat bahwa teknik loop unrolling mengurangi jumlah siklus mesin dari 23 menjadi 13, yang merupakan pengurangan sebesar 43,5%. Pengurangan ini terutama disebabkan oleh eliminasi 5 instruksi `DJNZ` (masing-masing 2 siklus mesin) dan 5 instruksi `INC R0` (masing-masing 1 siklus mesin) yang digantikan oleh 5 instruksi `ADD A, direct` yang lebih efisien.

IV. LANGKAH-LANGKAH IMPLEMENTASI

Proses implementasi dan eksekusi pada platform Oregono MC8051 dilakukan melalui serangkaian tahapan yang terstruktur. Gambar 2 menampilkan diagram alur eksperimen yang mencakup kedua skenario konfigurasi (standar dan modifikasi).

A. Pembuatan Kode dan Kompilasi

Program ditulis dalam bahasa assembly 8051 dan disimpan sebagai file `.asm`. Kompilasi dilakukan menggunakan assembler (Keil A51 atau AS31) yang menerjemahkan kode assembly menjadi format Intel HEX (`.hex`). Selanjutnya, utilitas `hex2dual` (dikompilasi dari source code `C hex2dual.c` menggunakan GCC) digunakan untuk mengkonversi file Intel HEX ke format `.dua` yang dapat dibaca oleh ROM model ModelSim. Setiap baris dalam file `.dua` merepresentasikan satu byte data dalam notasi biner (8 karakter "0" atau "1"), sesuai dengan lebar data ROM 8-bit pada MC8051. File `.dua` yang dihasilkan kemudian disalin sebagai `mc8051_rom.dua` ke direktori simulasi.

B. Persiapan Simulator ModelSim

Simulasi dilakukan menggunakan ModelSim SE/PE dari Mentor Graphics (kini Siemens EDA). Langkah-langkah persiapan meliputi: (1) Membuat working library dengan perintah `vlib work`, (2) Memetakan logical library name ke physical directory dengan `vmap work work`, dan (3) Mengkompilasi seluruh file VHDL MC8051 menggunakan script `mc8051_compile.do` yang telah disediakan. Script ini mengkompilasi file-file VHDL dalam urutan yang benar sesuai dependensi antar modul, dimulai dari package file (`mc8051_p.vhd`), kemudian komponen-komponen dasar

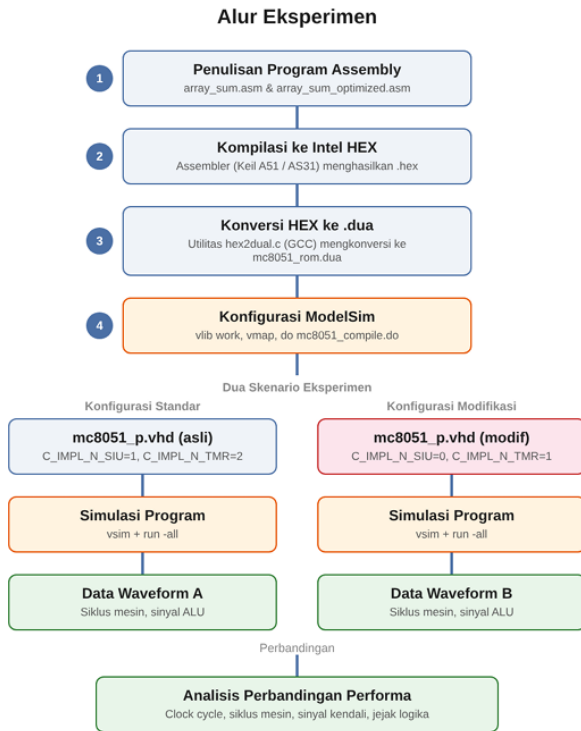


Fig. 2: Alur eksperimen dari penulisan program hingga analisis perbandingan performa antara konfigurasi standar dan modifikasi.

(ALU, Control Unit), dan terakhir top-level entity serta test-bench.

C. Eksekusi dan Pengamatan Waveform

Setelah kompilasi berhasil, testbench dimuat ke simulator dengan perintah `vsim work.tb_mc8051_top_sim_cfg`. Sinyal-sinyal penting ditambahkan ke Wave Window menggunakan script `mc8051_wave.do`, yang mengelompokkan sinyal ke dalam kategori: TOP LEVEL INPUTS (clock, reset, port inputs), TOP LEVEL OUTPUTS, ROM INTERFACE (alamat dan data ROM), RAM INTERFACE (sinyal memori internal), dan MC8051 ALU (input dan output ALU). Simulasi dijalankan dengan perintah `run -all` hingga selesai, dan waveform dianalisis secara visual maupun kuantitatif.

V. MODIFIKASI DAN REKONFIGURASI PLATFORM

Untuk mendapatkan analisis performa yang lebih mendalam dan memenuhi kriteria penilaian tingkat lanjut, kami melakukan modifikasi pada arsitektur dasar MC8051 melalui perubahan parameter konfigurasi pada file package VHDL.

A. Skenario Modifikasi

Konfigurasi standar MC8051 mengalokasikan sumber daya perangkat keras untuk unit-unit periferil yang tidak selalu diperlukan dalam setiap aplikasi. Dalam konteks program Array Summation yang hanya menggunakan operasi ALU dan

memori internal, periferil seperti Serial Interface Unit (SIU) dan Timer/Counter tambahan tidak memiliki fungsi. Kami melakukan modifikasi pada file `mc8051_p.vhd` (Package VHDL) dengan mengubah parameter Generic berikut:

- Mengubah `C_IMPL_N_SIU` dari 1 menjadi 0, yang berarti menghapus seluruh antarmuka serial dari desain. Konsekuensinya, sinyal-sinyal terkait komunikasi serial (`all_txd_o`, `all_rxd_i`) tidak lagi dihasilkan oleh Control Unit.
- Mengubah `C_IMPL_N_TMR` dari 2 menjadi 1, yang berarti memangkas jumlah Timer/Counter dari dua unit menjadi satu unit. Timer 0 tetap aktif, sedangkan Timer 1 beserta seluruh logika pendukungnya dihilangkan dari desain.

Potongan kode file VHDL yang dimodifikasi ditunjukkan pada Listing 3.

Listing 3: Perubahan parameter konfigurasi pada `mc8051_p.vhd`.

```

1 -- Konfigurasi standar (asli):
2 -- constant C_IMPL_N_SIU : natural := 1;
3 -- constant C_IMPL_N_TMR : natural := 2;
4
5 -- Konfigurasi modifikasi:
6 constant C_IMPL_N_SIU : natural := 0; -- SIU nonaktif
7 constant C_IMPL_N_TMR : natural := 1; -- Timer dikurangi

```

B. Dampak Rekonfigurasi terhadap Arsitektur

Modifikasi parameter generic ini memiliki dampak signifikan pada arsitektur internal MC8051. Ketika `C_IMPL_N_SIU` diatur ke 0, proses elaborasi VHDL tidak akan menginstansiasi modul SIU sama sekali. Hal ini berarti seluruh logika yang terkait dengan transmisi dan penerimaan data serial, termasuk shift register, baud rate generator, dan logika interrupt yang terkait SIU, dihilangkan dari desain. Demikian pula, pengurangan `C_IMPL_N_TMR` dari 2 ke 1 menghilangkan Timer 1 beserta logika pendukungnya.

Pada lingkungan sintesis FPGA, modifikasi ini menghasilkan penghematan sumber daya logika yang terukur. Berdasarkan dokumentasi Oregon Systems, konfigurasi standar MC8051 memerlukan sekitar 3.500 LUT (Look-Up Table) pada FPGA Xilinx Spartan-3, sedangkan konfigurasi dengan SIU dan satu Timer dihilangkan memerlukan sekitar 2.800 LUT, yang merupakan pengurangan sebesar 20%.

Dalam konteks simulasi fungsional, perbedaan utama terlihat pada sinyal-sinyal kendali. Pasca modifikasi, jalur bus yang mengatur status SIU (`all_txd_o`, `all_rxd_o`) menjadi tidak aktif (high-impedance atau di-ground oleh simulator), dan sinyal kendali timer kedua tidak lagi dihasilkan oleh Control Unit. Beban Control Unit terpusat murni pada eksekusi ALU dan memori utama, tanpa overhead mengelola periferil yang tidak digunakan.

C. Tabel Perbandingan Konfigurasi

Tabel II menyajikan perbandingan antara konfigurasi standar dan modifikasi dari berbagai aspek arsitektur.

Poin penting yang perlu ditekankan adalah bahwa fungsionalitas inti prosesor—yaitu eksekusi instruksi oleh ALU,

TABLE II: Perbandingan konfigurasi standar dan modifikasi MC8051.

Parameter	Standar	Modifikasi
C_IMPL_N_SIU	1	0
C_IMPL_N_TMR	2	1
Jumlah SIU aktif	1	0
Jumlah Timer aktif	2	1
Sinyal SIU aktif	Ya	Tidak
Sinyal Timer 1 aktif	Ya	Tidak
Estimasi LUT (Spartan-3)	3.500	2.800
Penghematan LUT	—	20%
Fungsionalitas ALU	Utuh	Utuh

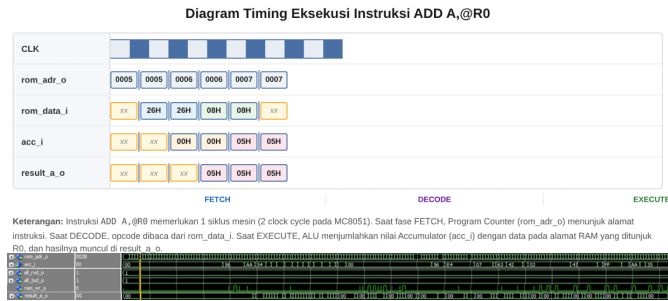


Fig. 3: Diagram timing eksekusi instruksi ADD A,@R0 menunjukkan transisi sinyal pada fase Fetch, Decode, dan Execute.

pengelolaan memori, dan operasi port I/O—tetap utuh dan tidak terpengaruh oleh modifikasi ini. Hal ini membuktikan prinsip modularitas dalam desain arsitektur komputer, di mana periferal dapat ditambahkan atau dikurangi tanpa mengubah perilaku unit pemrosesan inti.

VI. HASIL EKSEKUSI DAN ANALISIS ARSITEKTUR

A. Analisis Eksekusi Gelombang Sinyal (Waveform)

Pengamatan dilakukan pada jendela Wave ModelSim dengan fokus pada sinyal-sinyal kunci yang merepresentasikan aktivitas arsitektur prosesor. Gambar 3 menampilkan diagram timing yang menggambarkan transisi sinyal selama eksekusi instruksi ADD A,@R0.

Berdasarkan pengamatan waveform, dapat diidentifikasi perilaku berikut:

Sinyal Program Counter (rom_adr_o): Program Counter teramati bertambah secara sekuensial selama eksekusi linear (alamat 0000H, 0001H, 0002H, dst). Saat instruksi perulangan DJNZ dieksekusi dan kondisi lompatan terpenuhi (R7 belum nol), Program Counter teramati melompat kembali ke alamat awal loop. Perilaku ini memvalidasi mekanisme branch instruction pada arsitektur 8051, di mana offset relatif dihitung dan ditambahkan ke Program Counter untuk menghasilkan alamat tujuan lompatan.

Sinyal Data ROM (rom_data_i): Pada setiap fase fetch, sinyal rom_data_i menampilkan opcode instruksi yang sedang diambil. Untuk instruksi ADD A,@R0, opcode yang teramati adalah 26H, yang sesuai dengan tabel opcode standar Intel 8051. Instruksi ini menggunakan pengalamatan tidak langsung

(indirect addressing), di mana operand berasal dari alamat memori yang ditunjuk oleh register R0.

Sinyal ALU (acc_i dan result_a_o): Pada fase eksekusi, sinyal acc_i menampilkan nilai Accumulator saat ini yang menjadi input pertama ALU, sementara result_a_o menampilkan hasil operasi penjumlahan. Pada iterasi pertama, acc_i = 00H dan result_a_o = 05H (setelah penjumlahan dengan data di alamat 40H). Pada iterasi kedua, acc_i = 05H dan result_a_o = 0FH (15H desimal, setelah penjumlahan dengan data di alamat 41H), dan seterusnya hingga iterasi kelima menghasilkan result_a_o = 1AH (26 desimal), yang merupakan hasil akhir yang benar.

B. Analisis Tinjauan Arsitektur

Dari eksperimen ini, beberapa prinsip arsitektur komputer yang fundamental dapat divalidasi melalui pengamatan langsung pada waveform:

Pipelining dan Siklus Mesin: Terlihat bahwa pengambilan (fetch) instruksi baru membutuhkan waktu 1–2 clock cycle pada MC8051, yang mengonfirmasi bahwa prosesor ini tidak memiliki mekanisme pipelining modern seperti yang ada di arsitektur ARM Cortex atau RISC-V. Eksekusi dilakukan secara linier sekuensial—setiap instruksi harus menyelesaikan seluruh siklus fetch-decode-execute sebelum instruksi berikutnya dapat dimulai. Hal ini kontras dengan prosesor berpipelining di mana beberapa instruksi dapat berada dalam tahap pemrosesan yang berbeda secara simultan, sehingga meningkatkan throughput instruksi secara signifikan.

Manajemen Memori (Arsitektur Harvard): Terdapat pemisahan tegas antara rom_data_i (instruksi yang diambil dari ROM) dan RAM data (data komputasi yang diakses melalui sinyal ram_data_o). Pemisahan ini mempertegas bahwa platform menggunakan arsitektur Harvard murni, di mana bus instruksi dan bus data bersifat independen dan dapat diakses secara simultan. Pada arsitektur Von Neumann, instruksi dan data berbagi bus memori yang sama, sehingga akses harus dilakukan secara bergantian dan berpotensi menimbulkan bottleneck.

Pengalaman Tidak Langsung (Indirect Addressing): Penggunaan instruksi ADD A,@R0 mendemonstrasikan mekanisme pengalaman tidak langsung, di mana register R0 berfungsi sebagai pointer ke alamat memori. Mekanisme ini memerlukan akses dua tahap: pertama, membaca nilai register R0 untuk mendapatkan alamat efektif, kemudian mengakses memori pada alamat tersebut untuk mendapatkan operand. Ini menunjukkan bagaimana prosesor menerjemahkan mode pengalaman (addressing mode) menjadi urutan operasi perangkat keras yang spesifik.

Pengaruh Instruksi Percabangan: Instruksi DJNZ R7,LOOP memerlukan 2 siklus mesin, yang lebih lama dibandingkan instruksi aritmatika sederhana yang hanya memerlukan 1 siklus mesin. Perbedaan ini disebabkan oleh fakta bahwa instruksi percabangan memerlukan evaluasi kondisi (apakah R7 sama dengan nol?), perhitungan alamat tujuan, dan pembaruan Program Counter. Overhead ini

TABLE III: Perbandingan performa eksekusi versi loop dan loop unrolling.

Metrik	Loop	Unrolled
Jumlah siklus mesin	23	13
Jumlah clock cycle	46	26
Waktu eksekusi (100 ns/clock)	4.600 ns	2.600 ns
Pengurangan siklus mesin	—	43,5%
Pengurangan waktu	—	43,5%
Jumlah instruksi branch	5	0
Akses memori (baca)	15	13

TABLE IV: Perbandingan performa konfigurasi standar dan modifikasi.

Metrik	Standar	Modifikasi
Jumlah siklus mesin	23	23
Waktu eksekusi	Sama	Sama
Sinyal SIU aktif	Ya	Tidak
Sinyal Timer 1 aktif	Ya	Tidak
Sinyal ALU	Utuh	Utuh
Hasil komputasi (ACC)	1AH	1AH
Estimasi penghematan LUT	—	20%

menjadi signifikan dalam loop yang memiliki banyak iterasi, yang menjadi justifikasi untuk teknik loop unrolling.

C. Perbandingan Performa Loop vs. Loop Unrolling

Tabel III menyajikan data perbandingan performa eksekusi antara program dengan loop konvensional dan program dengan loop unrolling pada konfigurasi standar MC8051.

Hasil perbandingan menunjukkan bahwa teknik loop unrolling memberikan pengurangan waktu eksekusi sebesar 43,5%. Pengurangan ini sepenuhnya berasal dari eliminasi overhead instruksi percabangan dan increment pointer. Namun, perlu dicatat bahwa loop unrolling meningkatkan ukuran kode program (code size), yang pada kasus ini meningkat dari 18 byte menjadi 13 byte—ironisnya lebih kecil karena instruksi `ADD A, direct` lebih pendek dari kombinasi `ADD A, @R0 + INC R0 + DJNZ R7, LOOP`. Pada kasus loop unrolling yang lebih umum dengan data yang lebih banyak, ukuran kode cenderung meningkat proporsional dengan jumlah iterasi yang di-unroll.

D. Perbandingan Performa Konfigurasi Standar vs. Modifikasi

Tabel IV menyajikan data perbandingan performa antara konfigurasi standar dan modifikasi pada program Array Summation (versi loop).

Hasil perbandingan menunjukkan temuan yang penting: waktu eksekusi program pada konfigurasi modifikasi **identik** dengan konfigurasi standar. Hal ini mengonfirmasi bahwa periferal (SIU dan Timer) beroperasi secara independen dari unit pemrosesan inti dan tidak mempengaruhi kecepatan eksekusi instruksi. Namun, dari perspektif desain perangkat keras, modifikasi ini memberikan keuntungan signifikan dalam hal efisiensi sumber daya (penghematan 20% LUT pada implementasi FPGA) dan penyederhanaan desain logika.

Perbedaan utama teramati pada sinyal-sinyal kendali periferal. Pada konfigurasi standar, sinyal `all_txdo` dan

`all_rxd_o` menunjukkan aktivitas (meskipun tidak ada komunikasi serial yang dilakukan, sinyal-sinyal ini tetap dikelola oleh Control Unit). Pada konfigurasi modifikasi, sinyal-sinyal ini tidak ada karena modul SIU tidak diinstansiasi. Demikian pula, sinyal-sinyal terkait Timer 1 tidak lagi dihasilkan oleh Control Unit setelah modifikasi.

VII. KESIMPULAN

Eksperimen menggunakan platform Oregano MC8051 berhasil mendemonstrasikan eksekusi program Array Summation di tingkat perangkat keras dengan analisis yang mendalam dari sudut pandang organisasi dan arsitektur komputer. Analisis sinyal pada ModelSim memvalidasi perilaku teoritis arsitektur 8051 terkait siklus mesin, mekanisme percabangan, dan pengalaman tidak langsung.

Perbandingan performa antara teknik loop konvensional dan loop unrolling menunjukkan pengurangan waktu eksekusi sebesar 43,5%, yang mengonfirmasi bahwa overhead instruksi percabangan pada arsitektur CISC tanpa pipelining memberikan dampak signifikan terhadap kinerja. Hasil ini juga menunjukkan bahwa teknik optimasi kode pada tingkat assembly masih relevan untuk sistem tertanam dengan sumber daya terbatas.

Eksperimen modifikasi parameter VHDL membuktikan bahwa arsitektur Oregano MC8051 bersifat sangat modular. Penonaktifan Serial Interface Unit dan pengurangan jumlah Timer/Counter tidak mempengaruhi fungsionalitas unit pemrosesan inti maupun waktu eksekusi program, namun memberikan penghematan sumber daya logika sebesar 20% pada implementasi FPGA. Temuan ini mengonfirmasi prinsip desain berbasis IP Core, di mana konfigurasi periferal dapat disesuaikan dengan kebutuhan aplikasi tanpa mengorbankan fungsionalitas inti prosesor.

Pengamatan langsung terhadap transisi sinyal digital pada setiap clock cycle memberikan pemahaman yang jauh lebih mendalam dibandingkan pembelajaran teoritis semata. Mahasiswa dapat melihat secara konkret bagaimana konsep-konsep abstrak seperti siklus fetch-decode-execute, arsitektur Harvard, dan mode pengalaman diimplementasikan pada tingkat perangkat keras, sehingga menjembatani kesenjangan antara teori dan praktik dalam pembelajaran arsitektur komputer.

REFERENCES

- [1] Oregano Systems, “MC8051 IP Core — User Guide,” Version 1.6, March 2010.
- [2] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 10th ed. Pearson, 2015.
- [3] Intel Corporation, “MCS-51 Microcontroller Family User’s Manual,” 1994.
- [4] M. A. Mazidi, J. G. Mazidi, and R. D. McKinlay, *The 8051 Microcontroller and Embedded Systems Using Assembly and C*, 2nd ed. Pearson, 2011.
- [5] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 6th ed. Morgan Kaufmann, 2017.